

Module 16 Assignment

JavaScripts — Questions & Answers

1. What Variables & Scoping What is the difference between var, let, and const in JavaScript?

- **var:** Function-scoped (or globally scoped if outside a function). Can be re-declared and updated. Hoisted to the top but initialized as **undefined**.
- **let:** Block-scoped ({}). Can be updated but not re-declared within the same scope. Hoisted but not initialized (Temporal Dead Zone).
- **const:** Block-scoped. Cannot be updated or re-declared. Must be initialized at declaration. For objects/arrays, the reference cannot change, but properties/contents can be mutated.

2. Operators What is the difference between and ==? Give one example.

- **=** is the assignment operator – it assigns a value to a variable.
- **==** is the equality operator – it compares two values for equality after type coercion (converting types if they differ).

Example:

```
let x = 5;           // Assignment
console.log(x == '5'); // true (string '5' is coerced to number 5)
```

3. Loops What is the difference between a for loop and a for-of loop?

- **for loop:** Traditional loop with initialization, condition, and increment. Gives full control over iteration and works with index-based access. Can iterate over anything with a condition.

Example:

```
for (let i = 0; i < arr.length; i++) { console.log(arr[i]); }
```

- **for-of loop:** Iterates over iterable values (arrays, strings, maps, sets, etc.) directly, returning the values of each item, not the index. Simpler and more readable for iterating over collections.

Example:

```
for (let value of arr) { console.log(value); }
```

4. Functions What is an arrow function? Write a simple example.

- An arrow function is a shorter syntax for writing functions using **=>**. It has lexical **this** binding (doesn't have its own **this**, inherits from the surrounding scope) and cannot be used as a constructor.

Example:

```
const add = (a, b) => a + b;
console.log (add(3, 4));    // 7
```

5. Advanced Functions What are an IIFE, a recursive function, and a generator function? Write one key characteristic of each?

- **IIFE (Immediately Invoked Function Expression):** A function that runs as soon as it is defined.

Key characteristic: Creates a private scope to avoid polluting the global namespace.

Example:

```
(function() {
    console.log("Hello");
})();
```

- **Recursive function:** A function that calls itself.

Key characteristic: Must have a base case (termination condition) to prevent infinite loops and stack overflow.

Example:

```
function count(n) {  
    if (n > 0) count (n - 1);  
}
```

- **Generator function:** A function defined with function* that can be paused and resumed, yielding multiple values over time.

Key characteristic: Uses the yield keyword to return values one at a time and returns an iterator object with a next () method.

Example:

```
function* numbers() {  
    yield 1;  
    yield 2;  
}
```